

CORTEX_PROMPT_SNOWFLAKE

Build a Complete Data Engineering Pipeline with Cortex Code - Bookstore DW Edition

Give Cortex Code the prompts below in order. Each prompt builds on the previous one. By the end, you'll have a fully automated medallion architecture pipeline with transformation, quality gates, governance, anomaly detection, and a live dashboard — all in Snowflake.

Dataset

We're working with a Django-based bookstore dataset already loaded in Snowflake database `BOOKSTORE_DW`, schema `RAW`. The source system is `django_source`.

Existing Bronze Tables (BOOKSTORE_DW.RAW):

- **RAW_USERS** — ID (NUMBER), MOBILE (TEXT), EMAIL (TEXT), FIRST_NAME (TEXT), LAST_NAME (TEXT), IS_ACTIVE (BOOLEAN), CREATED_AT (TEXT/timestamp), _EXTRACTED_AT (TEXT), _SOURCE_SYSTEM (TEXT)
- **RAW_BOOKS** — ID (NUMBER), TITLE (TEXT), AUTHOR (TEXT), PRICE (FLOAT, stored in paise/cents e.g. 4999 = ₹49.99), CATEGORY_ID (NUMBER), IS_ACTIVE (BOOLEAN), CREATED_AT (TEXT/timestamp), _EXTRACTED_AT (TEXT), _SOURCE_SYSTEM (TEXT)
- **RAW_CATEGORIES** — ID (NUMBER), NAME (TEXT), _EXTRACTED_AT (TEXT), _SOURCE_SYSTEM (TEXT)
- **RAW_ORDERITEMS** — ID (NUMBER), ORDER_ID (NUMBER), PRODUCT_ID (NUMBER), QUANTITY (NUMBER), PRICE (TEXT, e.g. "2999.00"), _EXTRACTED_AT (TEXT), _SOURCE_SYSTEM (TEXT)
- **RAW_PAYMENTS** — ID (NUMBER), ORDER_ID (NUMBER), PAYMENT_ID (TEXT, e.g. "FREE_PAYMENT"), PAID (BOOLEAN), CREATED_AT (TEXT/timestamp), _EXTRACTED_AT (TEXT), _SOURCE_SYSTEM (TEXT)
- **RAW_USER_EVENTS** — ID (NUMBER), USER_ID (NUMBER), EVENT_TYPE (TEXT, e.g. "login", "signup"), TIMESTAMP (TEXT/timestamp), _EXTRACTED_AT (TEXT), _SOURCE_SYSTEM (TEXT)

Key Data Notes:

- Prices in RAW_BOOKS are stored as whole numbers (paise/cents) — divide by 100 for display
- Prices in RAW_ORDERITEMS are stored as text with decimals (e.g. "2999.00")

- There is NO separate orders table — order information is derived from RAW_ORDERITEMS (ORDER_ID) and RAW_PAYMENTS (ORDER_ID)
 - PAYMENT_ID values like “FREE_PAYMENT” indicate free/complimentary orders
 - All timestamps are stored as TEXT and need casting to TIMESTAMP_TZ
 - All tables have _EXTRACTED_AT and _SOURCE_SYSTEM metadata columns
 - Current data volume: ~15 users, ~20 books, ~18 categories, ~21 order items, ~6 payments, ~27 user events
-

Prompt 1: Foundation — Create Silver and Gold Schemas

The BOOKSTORE_DW database and RAW schema already exist with data loaded. Create two additional schemas for the medallion architecture: SILVER for cleaned and validated data, and GOLD for analytics-ready data.

 SQL

```
-- Create Silver and Gold schemas in BOOKSTORE_DW
CREATE SCHEMA IF NOT EXISTS BOOKSTORE_DW.SILVER;
CREATE SCHEMA IF NOT EXISTS BOOKSTORE_DW.GOLD;
```

Prompt 2: Change Data Capture Streams

Create CDC streams on all 6 RAW tables in BOOKSTORE_DW.RAW. Enable them to capture existing rows (APPEND_ONLY = FALSE, SHOW_INITIAL_ROWS = TRUE) so the pipeline can process the initial data load and future incremental updates.

Prompt 3: dbt Transformations

Generate a complete dbt project with two layers of models targeting BOOKSTORE_DW:

Silver staging models (materialized as tables in BOOKSTORE_DW.SILVER) for all 6 source tables:

- **stg_users** — Cast CREATED_AT to timestamp, compute signup_tenure_days, create full_name (FIRST_NAME || ' ' || LAST_NAME), extract email_domain, handle nulls in MOBILE/EMAIL/names

- **stg_books** — Cast CREATED_AT to timestamp, convert PRICE from paise to decimal (PRICE / 100.0 as price_amount), assign price_tier (Budget < 20, Standard 20–50, Premium > 50), join category name
- **stg_categories** — Simple clean pass-through with ID and NAME
- **stg_orderitems** — Cast PRICE to numeric (as unit_price), compute line_total (QUANTITY * unit_price), derive order-level aggregation fields
- **stg_payments** — Cast CREATED_AT to timestamp, derive is_paid boolean from PAID column, categorize payment_method from PAYMENT_ID (FREE, RAZORPAY, STRIPE, etc.)
- **stg_user_events** — Cast TIMESTAMP to proper timestamp, categorize event types (auth_events: login/signup, browse_events, purchase_events)

Gold analytics models (materialized as tables in BOOKSTORE_DW.GOLD):

- **dim_customers** — Customer dimension with loyalty tiers based on total orders, lifetime spend, average order value. Tiers: New (1 order), Regular (2–3), Loyal (4–5), VIP (6+)
- **dim_products** — Product dimension with book details, category name, price tier, and sales count
- **dim_categories** — Category dimension with book count and total revenue per category
- **dim_date** — Date dimension covering full range of dates in the data (use a date spine)
- **fct_orders** — Fact table derived from orderitems grouped by ORDER_ID with total_amount, item_count, joined with payment status
- **fct_order_items** — Detailed fact with each line item, book info, and pricing
- **fct_daily_revenue** — Daily revenue aggregation with breakdowns by category and payment status
- **fct_user_activity** — User activity metrics from events (login_count, days_active, last_active_date, engagement_score)

Include comprehensive tests in schema.yml — unique and not_null on all primary keys, relationship tests for foreign keys, and accepted_values tests for tiers and segments.

Prompt 4: Deploy dbt to Snowflake

Deploy the dbt project to Snowflake. Run `dbt build` to create all the Silver and Gold tables. Verify all tests pass.

Prompt 5: Validation Gates

Create 3 validation gate stored procedures in BOOKSTORE_DW.PUBLIC:

Gate 1 — validate_bronze(): Check all 6 RAW tables for null primary keys, empty tables, and duplicate primary keys. Return a JSON result with pass/fail and details per table.

Gate 2 — validate_silver(): Check referential integrity — every orderitem should reference a valid product, every payment should reference a valid order_id from orderitems, every user_event should reference a valid user. Check for null required fields and negative prices. Return pass/fail.

Gate 3 — certify_gold(): Verify Gold tables are populated, no null keys exist, tier/classification values are all valid (e.g. loyalty_tier IN ('New','Regular','Loyal','VIP')), and row counts are reasonable relative to Silver sources. Return pass/fail.

Prompt 6: Task DAG Orchestration

Create a task DAG in BOOKSTORE_DW.PUBLIC that orchestrates the pipeline. The flow:

1. Root task checks if RAW streams have new data (runs every 15 minutes via WHEN clause on stream)
2. Run validate_bronze() — only proceed if pass
3. Run dbt Silver models
4. Run validate_silver() — only proceed if pass
5. Run dbt Gold models
6. Run certify_gold()

Use task dependencies (AFTER clause) and conditional logic to stop the pipeline if a gate fails.

Prompt 7: Data Quality Monitoring

Set up data quality monitoring using Data Metric Functions:

- Attach system DMFs (NULL_COUNT, DUPLICATE_COUNT, ROW_COUNT, FRESHNESS) to key columns on RAW_USERS.ID, RAW_BOOKS.ID, RAW_ORDERITEMS.ID, RAW_PAYMENTS.ID
 - Create a custom DMF that validates all prices are positive (check RAW_BOOKS.PRICE > 0 and RAW_ORDERITEMS.PRICE > 0)
 - Set monitoring schedule to every 60 minutes
-

Prompt 8: Governance and Tagging

Create a governance framework with tags in BOOKSTORE_DW.PUBLIC:

- **pipeline_layer** — tag every table as BRONZE, SILVER, or GOLD
- **data_classification** — tag tables as PUBLIC, INTERNAL, or CONFIDENTIAL
- **pii** — tag columns containing PII (MOBILE, EMAIL, FIRST_NAME, LAST_NAME) as TRUE across all layers
- **data_domain** — tag tables by domain: USER, BOOK, CATEGORY, ORDER, PAYMENT, or EVENT

Apply tags to all tables across RAW, SILVER, and GOLD schemas.

Prompt 9: Snowpark Python Processing

Create 3 Snowpark Python stored procedures in BOOKSTORE_DW.PUBLIC:

bronze_profiler() — Profile all 6 RAW tables: row counts, null counts per column, distinct value counts. Store results in BOOKSTORE_DW.PUBLIC.PROFILING_LOG table.

silver_anomaly_detector() — Detect anomalies:

- Price outliers in stg_books using IQR method
 - Unusually large order quantities (> 5 items in a single line)
 - Users with events but no orders (potential drop-offs)
- Store results in BOOKSTORE_DW.PUBLIC.ANOMALY_FLAGS table.

gold_enrichment() — Compute RFM (Recency, Frequency, Monetary) scores for customers using quartile bucketing. Assign segments: Champion (high R+F+M), Loyal (high F), At Risk (low R, high F), Hibernating (low R+F+M). Store in BOOKSTORE_DW.PUBLIC.RFM_SCORES table. Also create BOOKSTORE_DW.PUBLIC.PIPELINE_SUMMARY with row counts across all tables in all three layers.

Prompt 10: Streamlit Dashboard

Create a Streamlit in Snowflake dashboard with 5 tabs:

Tab 1 — Revenue & Sales: Top-line metrics (total revenue, total orders, unique customers, avg order value) as metric cards. Daily revenue area chart. Revenue by category bar chart. Orders trend line chart.

Tab 2 — Customers: Customer metrics (total, avg lifetime spend, avg order value). Spend by loyalty tier bar chart. Tier distribution donut/pie chart. Detailed customer table.

Tab 3 — Products & Categories: Product metrics (total books, avg price, categories count). Top books by revenue table. Revenue by category horizontal bar chart.

Tab 4 — Operations: Order count, payment success rate. Orders by payment method bar chart. Payment status breakdown.

Tab 5 — Pipeline Health: RFM segment distribution bar chart. Anomaly summary by type. Pipeline row counts by layer (Bronze/Silver/Gold) bar chart. Data freshness from profiling log.

Use `@st.cache_data(ttl=300)` for all queries. All charts should use Altair or `st.bar_chart` with proper labels. Query from BOOKSTORE_DW.GOLD and BOOKSTORE_DW.PUBLIC tables.

Execution Notes

- All objects should be created in BOOKSTORE_DW database
- Use role ACCOUNTADMIN and warehouse COMPUTE_WH
- The RAW data is already loaded — no need for ingestion/Snowpipe setup
- dbt project should target BOOKSTORE_DW with profile using role ACCOUNTADMIN
- For dbt, use `profiles.yml` without password/authenticator fields (session auth)